Flint front ends — consistency, performance, security, Day-2 operations

Three ways to put one S3 bucket behind agent workloads, compared on four dimensions — six factors each (Performance carries a seventh, the workload envelope), scored 0 (weakest) – 5 (strongest) from the architecture docs, adversarially verified per approach. Pages 2–3 give the reason behind every number; page 4 maps availability, the multicluster-to-one-bucket contract, and where other industry implementations would shift the numbers. NFS-over-S3 is charted in both flavors (sec=sys dashed).

NFS-over-S3 flint-lite "hub" — shipped + drilled

One **hub pod** runs an NFS server; client pods mount it over the network with the ordinary kernel NFS client — nothing installed in the pod. The hub keeps the live tree on its own disk (a PVC) and asynchronously publishes whole files to an S3 bucket. Because every client talks to one server, it is the only approach with true shared-filesystem semantics. Its two flavors differ only in how NFS clients are trusted: **sec=sys** (today's deployment — the client self-declares its uid; the network is the only boundary) and **krb5p** (Kerberos — cryptographic per-user identity plus wire encryption; needs a KDC and a keytab on every client node; the implementation is in-tree but dormant, never wired — scored as designed). *Archetype: NFS gateway over object storage (AWS Storage Gateway, Nasuni-class).*

FUSE flint-fuse — designed only, deprioritized

No server. Each pod gets an injected **FUSE filesystem sidecar** that intercepts the app's file I/O, keeps data in the pod, and publishes that pod's workspace subtree straight to the same S3 bucket on a cadence. Each pod holds prefix-scoped bucket credentials, and needs a privileged sidecar (/dev/fuse). Designed in full; currently deprioritized in favor of Lean. *Archetype: bucket-native FUSE (Mountpoint for S3, gcsfuse, s3fs).*

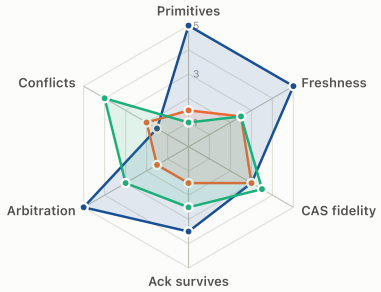
Lean flint-lean — built + chaos-drilled · cluster drill open

No server and no interception: the app works on **plain local files**. An ordinary, unprivileged sidecar (flint-sync) checks the workspace out of S3 at pod start and re-publishes changed files on a cadence through a small **gateway** that arbitrates who may publish; pods hold no bucket write credentials. All three approaches share one bucket format and are mutually convertible.

Consistency

NFS-over-S3 (either flavor) · FUSE · Lean

krb5p ≡ sec=sys here — Kerberos wraps authentication, not coherence



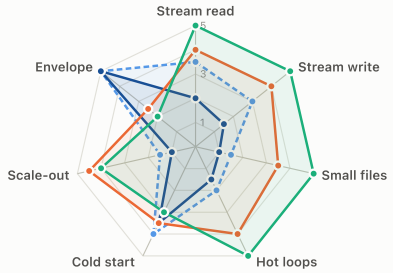
	NFS	FUSE	Lean
Primitives	5	1.5	1
Freshness	5	2.5	2.5
CAS fidelity	3	3	3.5
Ack survives	3.5	1.5	2.5
Arbitration	5	1.5	3
Conflicts	1.5	2	4

Consistency is arbitration, not proximity to S3: the hub sweeps primitives, freshness and arbitration because every operation takes the round-trip where ordering is manufactured. The inversion is the story — the hub is nearly weakest at foreign-write surfacing (the "three fates"), while Lean posts the best CAS and conflict scores: its arbitration is designed to turn detection into refusal — designed, not built: the Arbitration cell records the proxy as verified unbuilt, so today that refusal is a control-plane property — chaos-drilled: a deposed writer's zero citations survived, and the inbox/merge manifest never picks a silent winner on the designed door (a write placed straight into the bucket is

Performance

sec=sys · krb5p · FUSE · Lean

hot paths are local for the direct modes; measured numbers where they exist



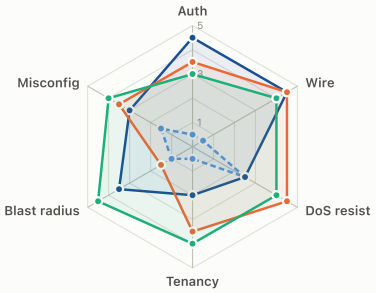
	sys	krb5p	FUSE	Lean
Stream read	3.5	2	4	5
Stream write	3	1.5	4	5
Small files	1.5	1	3.5	5
Hot loops	2	1.5	4	5
Cold start	4	3.5	3.5	3
Scale-out	1.5	1	4.5	4
Envelope	5	5	2.5	2

The consistency chart inverted: the direct modes win every hot-path axis because the data path is local — Lean's plain files sweep streaming, small-file and hot-loop I/O, while the hub's measured metadata cliff (create 12.5×, delete 222× slower) is its worst cell. The hub's one win is cold start against a live share (pod churn pays only a mount). krb5p trails sec=sys everywhere: GSS privacy encrypts every RPC byte. The path is implemented and interop-verified as of 2026-08-27, which is what makes this charge legitimate rather than hypothetical — but no krb5p throughput has been measured, so every cell here is still scored as designed. The

Security

sec=sys · krb5p · FUSE · Lean

scored as designed — krb5p implemented, interop-verified and enforceable since 2026-08-27



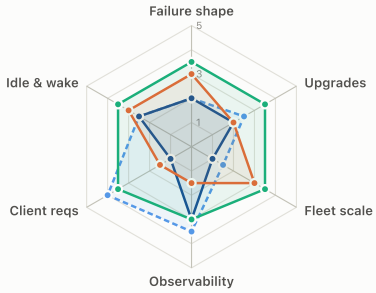
	sys	krb5p	FUSE	Lean
Auth	0.5	4.5	3.5	3
Wire	0.5	4.5	4.5	4
DoS resist	2.5	2.5	4.5	4
Tenancy	0.5	2	3.5	4
Blast radius	1	3.5	1.5	4.5
Misconfig	1.5	3	3.5	4

The widest spread: sec=sys sits at the bottom on five of six axes — identity self-asserted, wire cleartext, the boundary is reachability. krb5p buys the strongest auth and wire, and as of 2026-08-27 on a code path that is implemented, interop-verified against MIT krb5 and mounted rather than dormant; the misconfiguration charge it used to carry is largely paid off by a minimum-flavor floor that fails closed, though that floor still defaults to permissive. FUSE wins tenancy (unforgeable IAM principal tags) yet craters on pod compromise: credentials in every pod plus a privileged sidecar. Lean holds the best envelope — zero pod write credentials, seconds revocation, unprivileged sidecar. DoS resistance is quota-

Day-2 operations

sec=sys · krb5p · FUSE · Lean

what it takes to run each front end at fleet scale



	sys	krb5p	FUSE	Lean
Failure shape	2	2	3	3.5
Upgrades	2.5	2	2	3.5
Fleet scale	1.5	1	3	3.5
Observability	3.5	3	1.5	3
Client reqs	4	1	1.5	3.5
Idle & wake	2.5	2.5	3	3.5

Concentration costs the hub the most: both NFS flavors inherit share-wide hang-shaped failures (D-state clients, ~6 min force-detach) and a 3000-Service standing ladder — recurring fleet cost is priced inside Fleet scale (FUSE heartbeats ≈ $3.9k/mo at 3000 shares; Lean publish amplification ≈ 2.9 TiB/day for a hot 2 GiB file). krb5p subtracts further on every client-side axis (KDC, keytabs, rpc.gssd). Lean beats FUSE nearly everywhere by shedding privilege and the restart-in-place choreography; the hub keeps observability (rpoClean, /status) — and Lean's new wedge is a gateway/proxy outage: publishes pause AND checkouts, restarts, sync and HITL writes wedge.

Scores 0 (weakest) – 5 (strongest): a verified read of the architecture docs at HEAD (B5–B8 2026-08-25 pre-auth surface, strict / fuse / lean architecture pages, the NFS hardening plan and the lean plan of record. Method: one proposer agent per dimension, one adversarial verifier per approach column (six score adjustments adopted), one cross-dimension critic (double-counted facts re-homed; cold start re-anchored to a live share). Everything is scored as designed, with maturity shown separately: hub shipped + drilled · krb5p implemented to RFC 3961/3962/8009 + 4121 + 2203, interop-verified against MIT krb5 and mounted (sec=krb5p) with a minimum-flavor floor, but krb5i never mounted and no negative legs yet · FUSE designed only, deprioritized · Lean built through Phase 6 with five kind drill suites green (68 legs: chaos 12, bucket verbs 27, boundary 14, chart 8, operator 7); the real-cluster drill is written (3 legs) but has never been run. B1–B8 and B10 hardening fixes landed at HEAD are reflected; B12 prefix-reuse remains open.

Why these numbers — Consistency · Performance

Consistency

Strong cross-client primitives — Whether byte-range locks, O_EXCL create, atomic rename, and shared sqlite/git actually work between clients/pods, not just inside one.

- **NFS 5** — "opens, byte-range locks, O_EXCL and rename are real across every client of the share"; shared sqlite/git is shipped and cluster-proven. krb5p identical — GSS wraps the RPCs, not what they mean (flint-lite-consistency.html).
- **FUSE 1.5** — "no cross-pod locks, no shared sqlite / git — refused, not discouraged. ENOLCK and EXDEV"; one survivor: "O_EXCL as a synchronous If-None-Match:* PUT — the one strongly consistent cross-pod primitive" (flint-fuse-consistency.html).
- **Lean 1** — "PLAIN FILES — a real local filesystem, full POSIX, zero interception" — nothing can be made cross-pod, not even FUSE's O_EXCL; cross-pod is "snapshots + sync only"; sqlite/git single-pod only (flint-lean-architecture.html).

Read freshness across clients — How stale another client's view of your write can be — from enforced close-to-open down to snapshot-until-explicit-sync.

- **NFS 5** — Enforced close-to-open: "the hub is the one coherence authority"; only residuals are a held-open file's open()-time snapshot and readdir cached up to acdirmax 60s (flint-lite-consistency.html).
- **FUSE 2.5** — Another pod reads "a consistent snapshot at most one flush interval + one revalidation old; never read-your-peers'-writes" — bounded automatic staleness, between NFS close-to-open and Lean's explicit sync (flint-fuse-consistency.html).
- **Lean 2.5** — RESCORED 1.5 → 2.5: the design under the old number changed. It read “sync is harness-invoked… never background” and staleness “unbounded until explicit sync” — both now false. The run loop polls sentinels on its own clock; gated mode is REFUSED without `visibilityLagBoundSecs`, so citation staleness is bounded by construction (drill B10: the cap forces a citation mid-change); and `.flint/remote.seq` tells a reader there is news from a LOCAL file at zero bucket cost (drill B7: a credential-free probe container saw a foreign write land). Still below an enforced close-to-open: the reader is SIGNALLED, not refreshed — it must still pull. Level with FUSE for opposite reasons — FUSE refreshes without the app but cannot bound the window or name a coherent point; lean bounds and names one but needs the agent to act.

CAS / conditional-write fidelity — What If-Match / If-None-Match actually bind: which writer populations, detection vs refusal, cooperative vs enforced.

- **NFS 3** — v1.30 If-Match on PUT/DELETE/move — "a competing write answers 412 instead of silently losing your edit" — but detection between API callers, not exclusion against the mount; evict/hydrate flips ETags (fleets guide).
- **FUSE 3** — Verifier raised 2.5 to 3: publishes are 100% CAS-guarded ("If-Match on every publish") and "bucket policy enforces conditional writes (s3:if-none-match)" binds every bucket writer — unlike the hub's mount-exempt CAS (fuse-architecture).
- **Lean 3.5** — "the gateway is the enforcement point these cells always lacked — detection becomes refusal"; must-FAIL arms (stale If-Match MUST 412) and per-request epoch validation (flint-lean-architecture.html). Lowered 4 to 3.5 on a code read (2026-08-27): the two mechanisms this cell credited are not built. The sidecar has no HTTP client at all — warp, a server framework, is its only HTTP dependency — so it never speaks to the gateway; and inject.rs puts credentialsSecretRef on the sidecar container as envFrom, so pods do hold bucket write credentials. The architecture page itself files the zero-credentials phrase under "grade 2 — PLAN OF RECORD, not yet built", and the Arbitration cell one spoke over already says "Verified unbuilt".

Acked-write crash durability — Which failure domain an acked write/fsync survives before the write-back tier lands it in S3.

- **NFS 3.5** — "an acked write is crash-durable on the hub's PVC immediately" — survives hub crash, reschedule, node crash; only PVC loss converts RPO to loss. Not 5: fsync still does not mean S3 (flint-lite-consistency.html).
- **FUSE 1.5** — fsync ack is "durable to the pod's emptyDir... NOT the pod. NOT S3"; "the loss boundary is POD DEATH" — routine on spot; preStop drains ~9–15 GiB inside a 120s notice, a hard crash drains nothing (flint-fuse-consistency.html).
- **Lean 2.5** — RESCORED 1.5 → 2.5: the boundary-verbs campaign falsified all three clauses of the old reason (“RPO per pod; preStop drain best-effort… hard crash drains nothing”). The drain is now bounded-retry and sized against the ~2-minute spot ceiling — drill B11c: SIGTERM with a writer still running drained in 0 s and cited everything staged. A hard crash no longer drains nothing: uncited work is named durably in `orphans.json` IN THE BUCKET and `recover-staged` re-cites it from bucket truth alone — drill B11b: SIGKILL, fresh emptyDir, replacement pod recovered 8 orphans it could not have known about locally. And the verbs add a durability primitive that did not exist: an agent drops `.flint/publish` and the ack means the bytes are IN S3, not in the pod. Not higher, because an ordinary fsync still reaches only the emptyDir — lean’s durable ack is explicit and opt-in, where NFS’s is transparent.

Multi-writer arbitration — Who arbitrates concurrent writers to the same tree and what a second writer experiences — serialized, refused, or merely detected.

- **NFS 5** — Two writers, one file: "serialized — locks and share reservations are real" — the hub round-trip is where opens, locks and renames get ordered; a second writer blocks or is denied, never corrupts (flint-consistency-comparison.html).
- **FUSE 1.5** — Two writers on one file are "out of contract — one writer per subtree, lease-enforced"; the lease is cooperative CAS that "binds only writers that cooperate" — a credentialed non-cooperator is detected, not stopped (comparison doc).
- **Lean 3** — RESCORED 2.5 → 3: the axis asks what a SECOND WRITER EXPERIENCES, and the first scoring answered only the data-plane half. A deposed sidecar used to leave its agent waiting on an ack forever. Drill B18, measured: the zombie settles the owed ack as `refused-fenced` NAMING the winning epoch, flips `capabilities` to state=fenced/verbs=[], refuses a later sync sentinel, and leaves its tree hash unchanged — silence became an attributable refusal. Not higher because the data plane genuinely does merely detect: Enforced at the control plane and now measured: the chaos drill's deposed straggler self-fenced before its CAS and zero citations survived — but it landed 7,591 further data PUTs (uncited orphans); data-plane fencing awaits P5 at the proxy (chaos leg C3). Verified unbuilt 2026-08-26: no proxy component exists and the only per-request epoch check guards the gateway’s CONTROL endpoints, not the data path — so the split stands, control plane REFUSES (drill B18) while the data plane merely DETECTS (drill B12). P5 would make both arms refuse: estimated 3.5–4, not 5, since it excludes the loser rather than serializing concurrent writers.

Foreign-write conflict surfacing — The fate of an out-of-band/foreign write against a live tree: silently lost, undetected, or preserved-and-surfaced.

- **NFS 1.5** — S3-side writes vs a live hub: "three fates: silently destroyed by local-wins, undetected while resident, or adopted as truth by a hydration 412"; REST-vs-mount race: "the ETag will not save you" (flint-lite-consistency.html).
- **FUSE 2** — "same three fates as hub mode — destroyed, undetected, or adopted" — mitigated only structurally: the read-only project role and prefix-scoped IRSA narrow who can write foreign at all (flint-fuse-consistency.html).

Performance

Streaming read — Large sequential read throughput on warm/resident data, as the workload experiences it.

- **sys 3.5** — Measured 164 MB/s vs 147 MB/s local (1.1×) — "Streaming is fine; metadata is the constraint" — competitive per client but capped by the wire and the hub's single NIC (flint-lite-for-agent-fleets.md).
- **krb5p 2** — Privacy decrypts every READ payload per RPC. No longer a design estimate over a dormant path — the crypto is implemented, vector-pinned and interop-verified, and a sec=krb5p mount serves reads — but the throughput cost has never been measured, so it stays unquantified — below sys's measured 164 MB/s (pnfs-operator-runbook.md).
- **FUSE 4** — As designed: resident reads are "full local POSIX at disk speed", but "v1 ships the classic daemon data path" — every read byte transits the userspace daemon; passthrough/io-uring are opportunistic only (flint-fuse-architecture.html p3).
- **Lean 5** — "PLAIN FILES — a real local filesystem, full POSIX, zero interception" — reads come from the node's own disk and page cache with nothing of flint's in the path (flint-lean-architecture.html).

Streaming write — Large sequential write throughput on the application's write path (background publish excluded unless it blocks the app).

- **sys 3** — Measured: 512 MiB sequential write 104 MB/s vs 173 MB/s local — 0.6× of local on the app's write path (flint-lite-for-agent-fleets.md "What it performs like").
- **krb5p 1.5** — Every WRITE payload GSS-wrapped per message on the client and the hub's pure-Rust path; unquantified in docs — qualitatively below sys's measured 0.6×-of-local (pnfs-operator-runbook.md).
- **FUSE 4** — Writes land on the emptyDir working set at local speed minus the classic-FUSE copy path; the 8–13 s/GiB publish rate is background and never blocks the app's write (flint-fuse-architecture.html p3–p4).
- **Lean 5** — The app writes plain local files with zero interception; publish is an asynchronous cadence scan, measured 8.0 s/GiB sidecar-side, outside the write path (flint-lean-0b-measurements.md).

Small-file / metadata-heavy — Throughput of create/stat/delete storms and many-small-file trees (npm install, untar, large clones).

- **sys 1.5** — Measured create 12.5× slower (123/s), delete 222× — "each is a synchronous round trip — and that is what dominates the tools agents actually run"; the guide's own advice: move this to emptyDir (flint-lite-for-agent-fleets.md).
- **krb5p 1** — The same synchronous per-op round trips, each additionally GSS-wrapped and unwrapped; implemented and mounted, still unquantified — strictly below sys's measured 12.5×/222× cliff (pnfs-operator-runbook.md).
- **FUSE 3.5** — Own-subtree create/delete are local — no network RTT — but each op pays a FUSE transition plus "one note*() from every mutating op" into state.db; never measured (flint-fuse-architecture.html p3).
- **Lean 5** — Storms hit the local filesystem natively; the measured 100k-file leg puts all cost in the background barrier (65 s first publish, 1.85 s idle tick, 27 MiB manifest), not app-facing op latency (flint-lean-0b-measurements.md).

Hot-loop local I/O — Latency-sensitive synchronous loops — sqlite transactions, git operations, build dirs, agent scratch — where per-op latency, not bandwidth, paces the tool.

- **sys 2** — "npm install, a build tree, or git clone... will be paced by create/delete, not by bandwidth"; sqlite/git are correct through the mount but every lock/fsync/commit is a network round trip (flint-lite-for-agent-fleets.md).
- **krb5p 1.5** — Same RTT pacing plus per-message GSS crypto on each small synchronous RPC; implemented in-tree, overhead still unquantified — scored as the designed posture (pnfs-operator-runbook.md).
- **FUSE 4** — Single-pod sqlite/git run on the local working set at "disk speed"; fsync-heavy loops still pay per-op daemon transitions; cross-pod shared sqlite/git is refused with errno — a scoping choice, not a slowdown (fuse-architecture p3).
- **Lean 5** — Proven with real binaries: "git fsck --strict clean... sqlite integrity_check ok, all rows present" on plain local files across 3 publish barriers — the loop itself is never intercepted (flint-lean-0b-measurements.md e2e leg).

Cold start / first access — Time from a new pod starting against a live, warm share to a usable workspace; after-idle and hibernated wake are priced under Day-2 Idle & wake.

- **sys 4** — The working set persists on the hub's PVC, so agent pod churn pays only a mount — the best per-pod story against a live share; evicted files hydrate at a measured 131 s/GiB over default fan-out 6 (flint-lite-for-agent-fleets.md).
- **krb5p 3.5** — Same PVC-warm hub machinery, plus GSS context establishment in the mount path — "KDC, per-node keytabs, rpc.gssd" — an added first-access dependency — now exercised end to end (KDC, rpc.gssd, mount) but never timed, so still a design estimate (pnfs-operator-runbook.md).
- **FUSE 3.5** — Boot "imports manifest stubs, no bytes moved" — fastest time-to-first-file — but each touched byte pays hydration: 72.5 s/GiB measured pre-fan-out; 1000-pod correlated starts risk 503 storms (flint-fuse-architecture.html p3, p5).
- **Lean 3** — "cold start downloads the whole workspace; fine at GiB scale" — sequential-loop floor 3.3 s/GiB, 49.5 s for 100k files (loopback MinIO, labelled FLOOR by its own rig); every pod start pays the full checkout even against a live share. Bounded fan-out has since landed and its default is now 32, re-measured proxy-shaped — at a 25 ms round trip a 20k-file checkout runs 39.6 s → 21.3 s (−46%). The earlier 1M loopback figure (16 m 24 s → 7 m 05 s) is NOT fan-out evidence: on loopback fan-out is flat across 16/32/64 (7.57/7.59/7.67 s), so that win was MinIO’s single disk. Fan-out is a lever only when the path is round-trip-bound, and it stays a constant-factor win — the structural fact is unchanged: no warm PVC, so every pod re-downloads (lean 0b measurements; Tier 1 read-path measurements 2026-08-27).

Scale-out ceiling — How aggregate throughput grows as client/pod count grows, and what shared component caps it.

- **sys 1.5** — "Not a parallel filesystem. One hub is one pod and one NIC" — every client's bytes aggregate into one pod's NIC; graduating is the pNFS profile, not migration (flint-lite-for-agent-fleets.md "What this is not").
- **krb5p 1** — The same one-pod one-NIC ceiling, and the hub's CPU must GSS-wrap/unwrap every byte for every client — the aggregate cap arrives at NIC or crypto CPU, whichever saturates first (pnfs-operator-runbook.md).
- **FUSE 4.5** — No hub in the data plane — "the only always-on component is S3 itself"; throughput scales with pods against S3, bounded by the bucket: 503 SlowDown bursts and an N× re-fetch duplication tax (flint-fuse-architecture.html p1, p4).
- **Lean 4** — Verifier lowered 4.5 to 4: grade-1 proxy is the decided posture — "Proxy posture (decided): grade-1 primary" — every publish and checkout byte transits the proxy tier; grade-2 presigned direct-to-S3 is deferred (lean plan §2.2, §8).

Workload envelope — Which workloads an approach can admit at all - maximum working set, maximum unpublished dirty set, admissible shapes (shared-RWX, multi-pod sqlite/git). The other axes assume the workload fits.

- **sys 5** — Only the hub hosts working sets beyond node-local disk (PVC + tier evict and hydrate); no dirty-set cap - acked writes land on the PVC; the routing table sends shared-RWX, multi-pod sqlite/git and big-artifact writers here (fuse-architecture p5).
- **krb5p 5** — Same PVC + tier data plane, so the same envelope: TiB-scale working sets, no dirty cap, shared trees admissible -

Why these numbers — Security · Day-2 operations

Security

Client authentication strength — How strongly a writer must prove who it is before the store or server accepts its operations.

- **sys 0.5** — "Port 2049 is AUTH_SYS: the client asserts its own uid and the server takes it"; B9: "AUTH_SYS trusts wire uid 0, no squash, no in-process authz" — identity is self-declared, not proven (flint-lite-architecture.html).
- **krb5p 4.5** — Strongest mechanism in the tree, and the acceptance gate the verifier deducted for now exists: FLINT_NFS_MIN_SEC sets a minimum flavor, anything below it is refused with AUTH_TOOWEAK, and SECINFO advertises exactly what the floor will accept rather than inviting clients into a flavor the server would then refuse. The earlier deduction — "accepts AUTH_UNIX unconditionally alongside GSS, so wiring cannot gate acceptance" — no longer holds (sec_policy.rs; server_v4.rs).
- **FUSE 3.5** — SigV4/IRSA machine identity, unforgeable at the store: "every pod holds prefix-scoped credentials (IRSA)" and "IAM principal tags enforce it independently" — strong proof, workload-granular not per-user; designed (fuse-architecture).
- **Lean 3** — Verifier lowered 3.5 to 3: the decided v1 auth is a static bearer, not even per-workspace ("Auth v1: bearer only vs + SigV4 — open"); SigV4/TokenReview deferred — a weaker proof of identity than FUSE's per-pod IRSA (lean plan §9.3, §8).

Wire protection — Confidentiality and integrity of data and control traffic in transit.

- **sys 0.5** — Cleartext RPC: the shipped posture gives "no per-user authentication or wire encryption"; RPC-with-TLS (xprtsec=mtls) is only prospective, attractive once client kernels reach 6.5 (pnfs-operator-runbook.md).
- **krb5p 4.5** — Per-message GSS privacy plus integrity on every NFS RPC, mutually authenticated — "the strongest wire posture in the tree". No longer a paper path: the crypto is implemented to RFC 3961/3962/8009 with RFC 4121 per-message tokens and RFC 2203 §5.3 framing, pinned to published test vectors rather than self-round-trip, interop-verified against MIT krb5 fixtures committed in-tree, and exercised by a real mount -o sec=krb5p against a live KDC. Still unproven: krb5i is implemented and unit-tested but has never been mounted, and there are no negative legs yet — wrong keytab, expired ticket, replayed sequence number (nfs/krb; nfs/gss_framing, 2026-08-27).
- **FUSE 4.5** — All data-plane traffic is S3 over TLS — "plain S3, no flint server anywhere in the read path"; no cleartext NFS hop exists in this design (flint-lean-architecture.html p2, covering direct-mode FUSE and lean alike).
- **Lean 4** — Verifier lowered 4.5 to 4: no lean doc claims TLS anywhere; grade-1 proxy is the decided posture and carries every publish and checkout byte; grade-2 "bytes go pod to S3 DIRECT" is deferred (lean plan §2.2, §8).

Unauthenticated DoS resistance — How little a merely-reachable, credential-less peer can do to exhaust memory, state, or availability.

- **sys 2.5** — B5–B8 fixed at HEAD (state-table quota, slot cap, idle deadline, READ clamp), but the surface stays a full stateful NFSv4.1 machine on an open port — "no credential needed"; the defense is caps, not auth (nfs-server-hardening-plan.md).
- **krb5p 2.5** — Same binary, same port, same B5–B8 fixes and the same pre-auth TCP/session surface; no doc claims GSS shrinks the floodable surface — scored equal to sys (nfs-server-hardening-plan.md B5–B8).
- **FUSE 4.5** — No flint-operated door to flood: "the only always-on component is S3 itself" — the reachable data plane is S3, which refuses unsigned requests; the only cluster service is a create-time webhook (flint-fuse-architecture.html).
- **Lean 4** — The gateway is stateless ("no PVC, no epoch of its own, no state") and a successful DoS is availability-only — publishes pause, checkouts wedge; no B6-class state minting, though no B5–B8-style battery yet (flint-lean-architecture.html).

Tenancy isolation — Strength and enforceability of the boundary that keeps one tenant/project from reading or writing another's data.

- **sys 0.5** — "no auth on 2049 — the network is the boundary"; "nfsClientCIDRs cannot express a client in another cluster"; B12 prefix-reuse adoption open at HEAD — "B serves A's files, no race required" (nfs-server-hardening-plan.md B12).
- **krb5p 2** — Proven per-user identity makes POSIX modes bind to a real principal, and an export can now refuse anything below its floor rather than merely decline to advertise it. Held at 2 all the same: the floor does not touch what pins this axis — same per-prefix hubs, same open-at-HEAD B12 prefix-reuse adoption hole, same network-shaped cross-cluster boundary (nfs-server-hardening-plan.md B12).
- **FUSE 3.5** — "IAM scope ends at ws-a/" with principal-tag conditions as "a second, unforgeable layer"; the two-principal split keeps bucket-wide levers out of workspace policies — store-side and unforgeable, but designed-only (fuse-architecture).
- **Lean 4** — "Proxy tenancy: project-granular, dedicated bucket or prefix per project (DECIDED)"; claim identity with "foreign-on-reused-prefix must refuse" arms closes the B12 class; within-project residual accepted for v1 (lean plan §9.6, Phase 1).

Pod-compromise blast radius — How little an attacker who owns an agent pod gains (credentials, privileges, forgeable identity) and how fast that gain can be revoked - higher = smaller, faster-contained blast.

- **sys 1** — Any compromised pod reaching 2049 "can claim any uid" with no root-squash — root on every reachable share, no per-client credential to revoke; saving grace: "the bucket trusts one principal: the hub" (flint-lite-for-agent-fleets.md).
- **krb5p 3.5** — A compromised pod yields only that user's short-lived ticket — "KDC, per-node keytabs, rpc.gssd": keytabs live on nodes, and the KDC can disable the principal; bounded to one identity. Now drilled against a real MIT KDC and a real Linux client rather than assumed, but in a throwaway VM realm, never in production (pnfs-operator-runbook.md).
- **FUSE 1.5** — Creds narrow cross-tenant reach "while irreducibly widening what a compromised pod can do to its own subtree"; the pod carries a "privileged · Bidirectional · /dev/fuse" sidecar — node reach; IRSA revocation slow (fuse-architecture).
- **Lean 4.5** — "pods hold ZERO bucket write credentials" — verified 2026-08-26 to be the AGENT container specifically: the webhook stamps the credential secret onto the SIDECAR only, the agent container carries no env and no envFrom, and shareProcessNamespace is false, so a compromised agent cannot reach it in either deployment mode. Proxy-mediated deployments hold none in the pod at all, and only they get the "kill the token" revocation speed and proxy-enforced prefix scoping; with credentialsSecretRef the scope is whatever IAM policy you attach, the sidecar an ordinary UNPRIVILEGED process — no /dev/fuse, no PSA conflict; "revocation in seconds — kill the token". The chaos drill locates data-plane enforcement at the PROXY: barriers bypass the gateway today (legs C8/C12).

Misconfiguration resistance — Whether the security posture fails closed and survives operator error - secure defaults, unforgeable second layers, refusal over silent degradation.

- **sys 1.5** — Verifier raised 1 to 1.5: the flagship footgun is fixed at HEAD a8a1f5a — SECINFO now advertises AUTH_SYS first / AUTH_NONE last, order pinned by a unit test, and "a stock mount now negotiates sec=sys" (compound.rs).
- **krb5p 3** — A refusal policy now exists and fails closed: FLINT_NFS_MIN_SEC is validated at startup and an unrecognised value refuses to boot rather than resolving to "no floor", so a typo cannot leave an operator believing enforcement is on while sec=sys is served; the advertisement is derived from the accept rule, so the two cannot drift. Not higher, because the floor still defaults to permissive and must be set deliberately, and KDC/keytab/rpc.gssd wiring remains classic sprawl with no CRD field, Helm value or other in-tree tooling to set it (sec_policy.rs, 2026-08-27).

Day-2 operations

Failure shape — How component failures present to running workloads: blast radius, hang vs error vs bounded loss, and the recovery ceremony they demand.

- **sys 2** — Hub NODE crash is the worst shape on any page: "every client hard-hangs in D-state; Recreate + RWO + ~6 min force-detach; F29's manual arm" — sleep SIGKILL cannot touch; benign pod-crash (13s, park and resume) offsets (fuse-arch p4).
- **krb5p 2** — Same hub concentration, so the identical D-state/force-detach class applies unchanged — and it adds KDC/keytab/rpc.gssd as further mount-path dependencies — wiring that is now exercised rather than prospective; scored as designed (pnfs-operator-runbook.md).
- **FUSE 3** — "a dead mount is ENOTCONN — an error, never a D-state hang"; failure distributes into "frequent, per-pod, LOSS-shaped events" — errors beat hangs, but the loss class is permanent and hub mode does not have it (fuse-architecture p4).
- **Lean 3.5** — "gateway down: publishes pause... far softer than hub-down; reads never depended on it" — "the softest failure shape" on these pages; held to 3.5: an outage also wedges checkouts, restarts, sync and HITL writes (lean plan §7).

Upgrades & restart recovery — Cost and risk of rolling new versions and recovering from process/container restarts under live workloads.

- **sys 2.5** — Measured-benign: "restart + epoch re-claim ≈ 13s; clients park and resume — nothing durable at risk (PVC)", MDS roll ~40s; docked for single-replica Recreate and G15's trap: a schema bump makes rollback a crash-loop (hardening plan G15).
- **krb5p 2** — Everything the sys hub carries, plus per-node keytabs and rpc.gssd join every client-node lifecycle event; the stack is implemented and mounted as of 2026-08-27, but no roll of it has ever been exercised; scored as designed (pnfs-operator-runbook.md).
- **FUSE 2** — "upgrade = pod churn: injection is create-time only... a flint-fuse CVE reaches the fleet by pod churn"; restart is fragile — "the reflex... destroys exactly the dirty set" — plus the 80–110s lease tax on unclean death (fuse-arch).
- **Lean 3.5** — Container restart is first-class in the shipped matrix — "never re-materialize: reload the persisted baseline, rescan to rebuild dirt, self-recognize the lease" — battery-tested; unprivileged sidecar lowers upgrade stakes (lean plan §2.1).

Fleet-scale standing footprint — Standing objects, control-plane load, and recurring coordination cost when the share count reaches thousands.

- **sys 1.5** — "hub: 3000 Services (73% of a /20 CIDR) · 3000 PVCs · the ladder"; cold start at 3000 CRs was a deterministic OOMKill (B3); rate-term blockers are fixed but the per-share standing-object ladder is by design (flint-lite-fleet-scale-plan.md).
- **krb5p 1** — Inherits the hub's entire 3000-object ladder and adds a KDC plus keytab distribution to every client node — standing security state sys does not carry, in every consumer cluster (pnfs-operator-runbook.md).
- **FUSE 3** — "mode: direct — NOTHING standing... one webhook, one bucket", but the page prices its own residue: "epoch heartbeats: 3000 cells at 10s defaults ≈ $3.9k/mo" pending lease-only-while-dirty; 1000-pod reclaim LIST risk (fuse-architecture p2).
- **Lean 3.5** — "Idle = S3 only, structurally. Lease cells exist only while a writer runs" — nothing standing per share, one stateless gateway; costs per-activity: a hot 2 GiB file ≈ 2.9 TiB/day at 60s, "bounded by policy, not physics" (lean plan §6).

Fleet observability — Ability to answer 'is it healthy, is it clean, what is the RPO' per share from an operable fleet-wide surface.

- **sys 3.5** — Best-instrumented of the four: /status "tells importing from wedged" with recovery point, epoch holder, client list; hibernate is "verify-then-delete, never assume" on rpoClean; docked for the silent wrong-PV-address trap (fleets guide).
- **krb5p 3** — Same /status + CRD surface, but the GSS path has no operational instrumentation — validated by a real MIT KDC mount and by pynfs --security=krb5, neither of which adds instrumentation — so keytab/context/ticket failures arrive as a new uninstrumented diagnosis class (pnfs-operator-runbook.md).
- **FUSE 1.5** — Deletes the fleet's best witness: "FUSE mode has no equivalent witness [to rpoClean], so the bucket... becomes the only global status surface"; per-pod metrics merely owed; loss rate "must be measured there" (fuse-architecture p4).
- **Lean 3** — Phase 3 names deliverables — "per-pod RPO gauge... publish-failure and checkout-wedged alerts, conflict-report surfacing" — and sync emits a machine-readable conflict report; the gauges shipped and are drilled — gauges.json plus a /metrics rendering off the same struct — but the named alert rules are still owed, and there is no client list (lean plan §5).

Consumer footprint — What must be installed, privileged, or configured on client nodes and pods for the data path to keep working day to day.

- **sys 4** — "Nothing from flint installed" — the data path is the node kernel's NFS client; consumers never get bucket credentials. Residue is config traps: without nconnect 2+ the kernel "silently refuses every additional trunk" (lite-architecture).
- **krb5p 1** — "Wiring it up (KDC, per-node keytabs, rpc.gssd, sec=krb5p mount opts)": every client node gains a keytab, a daemon and a realm dependency — the heaviest consumer requirement of the four — and now a real requirement rather than a prospective one (pnfs-operator-runbook.md).
- **FUSE 1.5** — The injected sidecar is "privileged · Bidirectional · /dev/fuse" in every matched pod; "PSA baseline / restricted refuse privileged sidecars" — forcing a node-broker endgame; kernel floor of plain fuse3 (flint-fuse-architecture.html p1/p2).
- **Lean 3.5** — Sidecar is an "ordinary process — UNPRIVILEGED — no /dev/fuse... no PSA conflict", kernel floor "drops to nothing", zero pod bucket credentials; under sys: native sidecars (K8s 1.29+) and webhook injection are mandatory (lean-architecture).

Idle lifecycle & wake — Cost of a parked share and the safety and latency of winding down and waking back up.

- **sys 2.5** — Two-rung ladder gives disk-only then S3-only cost, verified drain (wake ~41s / ~79s), but suspendWithSessions "defaults to suspending anyway" and suspending under a live hard mount is "not data loss; an indefinite hang" (fleets guide).
- **krb5p 2.5** — The ladder and its hazards are unchanged — the GSS layer rides the same :2049 data path and the runbook's Kerberos section states no interaction with park/wake; scored as designed (pnfs-operator-runbook.md).
- **FUSE 3** — "idle = Hibernated, structurally" — "the idle LADDER becomes vacuous, and with it the B6-class inversions"; the price: "every pod start is a Hibernated-class cold start" plus heartbeat economics (fuse-architecture p2).
- **Lean 3.5** — "Idle = S3 only, structurally. Lease cells exist only while a writer runs" — no ladder, no suspend hazard, no keepalive to get wrong; cold start is a full checkout through the proxy whose rate acceptance is still open (lean plan §6).

Availability — what still works when a component dies

The radar charts price failure as a score; this matrix shows its shape. Verdicts: ● unaffected ● degraded ● impaired ● outage. Both NFS flavors share one column — failure shape is identical; the KDC row exists only for krb5p.

	NFS-over-S3 (hub — both flavors)	FUSE	Lean
What the data path depends on	The hub pod, its PVC, and the network path to :2049 — for every read and every write.	The in-pod FUSE daemon for every I/O; S3 for non-resident reads and all publishes.	Nothing standing — the app reads and writes plain local files; S3 / gateway are needed only at checkout and publish.
Serving process crashes (hub pod / FUSE daemon / sync sidecar)	● **Brief stall** — restart + epoch re-claim ≈ 13 s; clients park and resume — nothing durable at risk (PVC).	● **Visible errors** — ENOTCONN — an error apps can see; restart-in-place recovers the emptyDir dirty set; open fds stay severed, apps reopen.	● **App unaffected** — plain files — the app never notices; the restarted sidecar reloads its baseline, rescans, self-recognizes its lease; publishes pause one beat.
Node hosting it dies (or spot-reclaims uncleanly)	● **Share-wide hang** — every client hard-hangs in D-state (SIGKILL cannot touch it); Recreate + RWO ≈ 6 min force-detach; F29's manual arm. Acked data is safe on the PVC.	● **Per-pod loss** — that pod's unflushed set is gone (hot files: unbounded) plus a 60–110 s subtree write-lockout for the successor; every other pod unaffected.	● **Per-pod loss** — a hard crash drains nothing — loss = RPO exactly; the successor waits out the same lease lockout; every other pod unaffected.
S3 outage or throttle	● **Serving unaffected** — reads and writes continue from the PVC; the RPO grows; one coordinated retry queue drains the backlog.	● **Degraded + compound risk** — non-resident reads fail; N uncoordinated engines re-fetch N×; outage + spot reclaim compound — that pod's dirty set is gone.	● **Running pods unaffected** — local files keep serving and publishes pause — but new pods cannot check out until S3 returns.
Control plane down (operator / webhook / gateway + proxy)	● **Serving unaffected** — running hubs keep serving; wake and reconcile are blocked — a suspended share cannot wake while no operator reconciles.	● **New pods blocked** — failurePolicy: Fail — every matched pod CREATE blocks while the webhook is down; running pods unaffected (Ignore would be worse: a silent data black hole).	● **Four wedges — at the proxy** — proxy unreachable: publishes pause AND checkouts/restarts wedge AND sync is unavailable AND HITL writes fail loudly (chaos leg C12). Gateway down alone: barriers continue — the shipped sidecar writes its cells straight to the store (C8); running pods keep serving throughout.
KDC / Kerberos infrastructure down — krb5p flavor only	● **New mounts fail** — new GSS contexts and credential renewals fail; established tickets ride out their lifetime; sec=sys is untouched (as designed — the GSS path is dormant).	**n/a** — no NFS wire.	**n/a** — no NFS wire.
Share idle / parked, then accessed	● **Wake required** — an NFS mount against a scaled-to-zero hub hangs until something writes the wake annotation — the data path cannot; the file API answers 503 fast; wake ≈ 41 s suspended / 79 s hibernated.	● **Nothing to wake** — idle = Hibernated structurally; the price moves to every pod start being a cold start — hydration on touch.	● **Nothing to wake** — idle = S3-only structurally; the price is a full checkout at pod start, budgeted by derived probes.

The pattern the docs themselves draw: the hub **concentrates** failure into rare, share-wide, hang-shaped outages; the direct modes **distribute** it into frequent, per-pod, loss-shaped events — "durability becomes an operational statistic — interruption handlers, grace periods, dirty-set caps — instead of an architecture property." Lean's refinement over FUSE is taking the app out of the sidecar's blast radius entirely; its residual concentration point is the gateway/proxy pair.

Beyond flint — the same archetypes in the industry

These pages are grounded in flint, but the columns model archetypes, and most orderings are structural — set by where arbitration, bytes, and credentials live — so they survive across implementations: the hub column reads on any NFS-gateway-over-object product, FUSE on the bucket-native class, Lean on any checkout-and-sync pattern. The measured decimals (the 12.5×/222× metadata cliff, hydrate and checkout rates) and the hardening state are flint's own. Where other implementations genuinely move numbers:

The FUSE column is the serverless, bucket-native class. Metadata-server FUSE filesystems (JuiceFS, Alluxio, CephFS-FUSE) are a different animal: a coherence authority returns, so on the Consistency chart they land near the NFS polygon (real locks, close-to-open) while keeping FUSE's privilege posture, per-pod failure shape, and Day-2 costs.

fsync semantics are a dial, not a constant. Mountpoint for S3 completes the upload on fsync/close — "Ack survives" rises toward 4 while streaming-write and hot-loop scores fall; flint's emptyDir ack is the write-back end of the same dial, chosen for latency.

Lean's CAS and conflict scores are the well-engineered end of sync. A plain aws-s3-sync-style loop does silent last-writer-wins (conflict surfacing ≈ 1, CAS ≈ 0), and most bucket-native FUSE clients send no conditional writes at all. The sec=sys and krb5p rows, by contrast, are textbook industry NFS facts and transfer unchanged.

lakeFS is the closest whole-system analogue — the same doctrine of one bucket format, several front ends: its S3 gateway is lean's grade-1 proxy, its presigned direct mode is the grade-2 lean deferred, lakectl local is checkout-and-sync, and lakeFS Mount ("Everest", enterprise; NFS v3 or FUSE, write support since 0.2.0, a CSI driver in preview) adds a mount front end — though its NFS is local snapshot delivery, not a coherence authority. The contrasts that matter: visibility is explicit-commit, not cadence ("other users or mounts will not see your changes until those changes are committed"); write-mode POSIX is narrower than plain files (no rename, links, or locks — rename alone disqualifies git loops in-mount) and same-branch concurrent mounts resolve *source-wins* — a silent winner. Against that, its server-side arbitration — atomic branch commits, first-class merge conflicts, per-user RBAC, clients holding no bucket credentials — is roughly lean's designed endgame already shipped, at the price flint-lean refuses to pay: an operated KV store (Postgres/DynamoDB) standing beside the bucket.

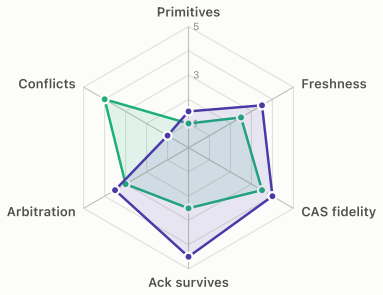
Sources: fuse-architecture p4 (failure matrix) · lean plan §2.2/§7 (gateway outage) · flint-lite-architecture p4 + fleets guide (idle/suspend) · pnfs-operator-runbook (Kerberos).

lakeFS — scored from source

Scored from source: a shallow clone of github.com/treeverse/lakeFS at 4bb11638 (2026-08-16), read by a 14-agent pass — 5 subsystem recon readers, 4 dimension scorers, 4 adversarial verifiers demanding file:line evidence, 1 comparison critic. Reconciliation: misconfig lowered by a verifier, CAS raised and upgrades trimmed by the critic, failure shape lowered 4→3 in post-publication review. One column, two postures — server API/gateway cells vs lakectl-local cells (each note says which face earns what). Everest mount (NFS/FUSE) is enterprise and absent from this repo.

Consistency

lakeFS overlaid on Lean — its nearest archetype; full scores in the table

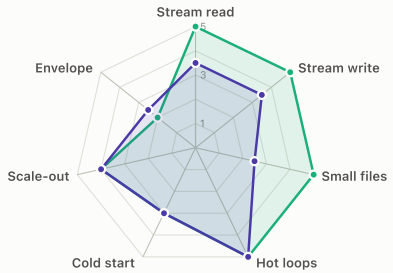


	NFS	FUSE	Lean	lakeFS
Primitives	5	1.5	1	1.5
Freshness	5	2.5	2.5	3.5
CAS fidelity	3	3	3.5	4
Ack survives	3.5	1.5	2.5	4.5
Arbitration	5	1.5	3	3.5
Conflicts	1.5	2	4	1

lakeFS replaced the filesystem with a KV-CAS commit store: acked writes are already in S3 + KV before the app sees success (4.5 vs Lean 2.5, graveler staging), API readers see staged writes immediately, and branch/merge machinery arbitrates with per-key conflict refusal (graveler.go:2289). The filesystem axes invert — no locks, no rename anywhere in the API, no shared sqlite/git — and foreign S3 writes are the mirror of Lean's inbox: hidden, served corrupt under a stale ETag, or GC-deleted (1 vs 4). CAS raised 3.5→4 by the critic: refusal-grade KV insert-if-absent outweighs the missing object-level If-Match.

Performance

lakeFS overlaid on Lean — its nearest archetype; full scores in the table

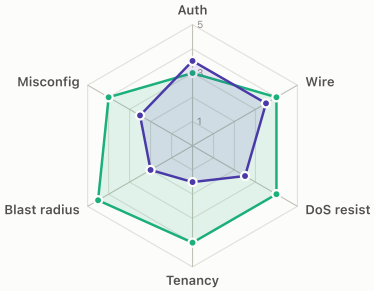


	sys	krb5p	FUSE	Lean	lakeFS
Stream read	3.5	2	4	5	3.5
Stream write	3	1.5	4	5	3.5
Small files	1.5	1	3.5	5	2.5
Hot loops	2	1.5	4	5	5
Cold start	4	3.5	3.5	3	3
Scale-out	1.5	1	4.5	4	4
Envelope	5	5	2.5	2	2.5

An envelope of two postures no single workload gets at once: hot-loop 5 and cold start belong to lakectl-local checkouts (plain files, nothing on the I/O path), while metadata-heavy work pays 2–3 HTTPS round-trips per file (2.5 vs Lean's 5). Streaming rides presigned-direct to S3; scale-out is stateless replicas until the KV store binds (4). Envelope 2.5: every POSIX workload needs a full local copy the repo's own CLI warns is non-scalable.

Security

lakeFS overlaid on Lean — its nearest archetype; full scores in the table

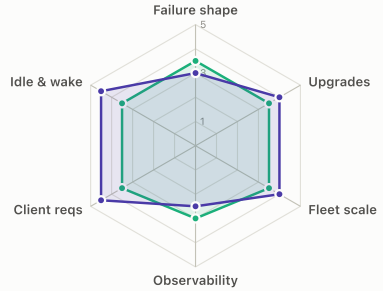


	sys	krb5p	FUSE	Lean	lakeFS
Auth	0.5	4.5	3.5	3	3.5
Wire	0.5	4.5	4.5	4	3.5
DoS resist	2.5	2.5	4.5	4	2.5
Tenancy	0.5	2	3.5	4	1.5
Blast radius	1	3.5	1.5	4.5	2
Misconfig	1.5	3	3.5	4	2.5

The score rests on SigV4 per-request signing at the gateway plus presigned URLs keeping bucket credentials server-side — OIDC/SAML/RBAC are enterprise (pkg/auth/build.go Fatals non-basic modes), so shipped OSS is effectively one superAdmin: tenancy 1.5 is where the split bites. Misconfig lowered 3→2.5 by the verifier: blockstore.signing.secret_key defaults to a publicly-known constant in ALL modes (config/defaults.go:107). Default-on hooks put writer-authored webhooks on the server's network position.

Day-2 operations

lakeFS overlaid on Lean — its nearest archetype; full scores in the table



	sys	krb5p	FUSE	Lean	lakeFS
Failure shape	2	2	3	3.5	3
Upgrades	2.5	2	2	3.5	4
Fleet scale	1.5	1	3	3.5	4
Observability	3.5	3	1.5	3	2.5
Client reqs	4	1	1.5	3.5	4.5
Idle & wake	2.5	2.5	3	3.5	4.5

Cheapest fleet citizen per share: a parked repo is KV rows plus bytes (idle 4.5), consumers need only a CLI/SDK (4.5), rolls are stateless restarts. Failure shape lowered 4→3 on review: server/KV death stops the whole data plane for every API/gateway consumer fleet-wide — only lakectl-local working files ride it out — where Lean's running pods keep serving. And the axes don't price the structural contrast: Lean operates ZERO databases (the bucket is the DR story); lakeFS carries an operated KV (refs + all uncommitted staging) plus an external Spark GC job. Observability 2.5: no per-repo metrics dimension; /_health checks neither KV nor blockstore.

lakeFS beats Lean exactly where the server owns the truth — an acked write is already in S3 plus the KV store before the app sees success (4.5 vs 2.5), API readers see each other's staged writes immediately (3.5 vs 2.5), and branch/merge machinery genuinely arbitrates concurrent writers with per-key conflict refusal (3.5 vs 3). The structural reason: it replaced the filesystem with a commit store.

Everything filesystem-shaped inverts. No locks, no rename, no shared sqlite/git across clients (1.5); metadata storms pay HTTPS round-trips against Lean's local-NVMe 5; every POSIX workload needs a full local checkout. The mount that would fix this — Everest, NFS/FUSE write mode — is enterprise and not in this codebase.

On out-of-band writes the two systems are exact mirrors: Lean preserves and surfaces foreign S3 writes through its inbox (4); lakeFS hides them, schedules them for GC deletion, or serves a foreign overwrite corrupt under an unverified stored ETag (1). The object store is lakeFS's private heap, not a shared surface.

As a fleet citizen lakeFS is cheaper than Lean per share — parking is free, consumers featherweight, rolls stateless — but it concentrates what Lean distributes: server/KV death stops the whole API data plane fleet-wide (failure shape 3 vs 3.5), OSS multi-tenancy is one superAdmin (tenancy 1.5 vs 4), and the default-on hooks engine puts writer-authored synchronous webhooks inside the commit path (pkg/actions, 6.4k LOC — flagged by the critic from a subsystem no scorer had read).

Method: recon → score → adversarial verify (file:line evidence, 452 tool calls) → cross-dimension critique; reconciliation adopted the verifier's misconfig adjustment and the critic's CAS raise, upgrades trim, and two-postures caveat. Evidence basis differs from pages 1–3: the flint columns are doc-grounded and drill-proven; this column is code-inferred from one commit (4bb11638) of the OSS repo — Everest mount, RBAC, and OIDC/SAML are enterprise deltas noted per cell. Radar overlays show Lean vs lakeFS; the tables carry every column. Cross-references to Lean cells on this page were re-synced to the live scores on 2026-08-27 — three had been quoting pre-rescore values, which is the hazard of hand-writing one page against another page's numbers. Post-publication review lowered Day-2 failure shape 4→3: the two-postures envelope had credited the API/gateway face with lakectl-local's outage behavior.

Multicluster — many clusters, one bucket

The radar scores on page 1 anchor to the single-cluster case — clients co-located with the share's cluster. This page states each approach's multicluster contract and lists the cells that move when clients span clusters. The direct modes and lakeFS are topology-invariant by construction; the hub column is where multicluster costs land.

Hub: coherence survives trivially (one authority) — the contract is network identity. Remote clusters SNAT to one address (1486/1486 measured; nfsClientCIDRs structurally cannot work), per-node NFS client names must be unique across clusters (RFC 8881 case 5 read a colliding name as a reboot and took the incumbent's locks — six 1.37.0 defects, one cause), and remote mounts need a keepalive or idle-suspend kills them. Prefix uniqueness stops at the cluster boundary — equal prefixes contend on the epoch cell, nested ones never meet: allocate prefixes upstream of Kubernetes.

FUSE / Lean: native — the bucket is the rendezvous and the arbitration state (epoch, claim, lease, manifest CAS, lean's gateway refusal) is store-side, so cross-cluster behaves exactly like cross-pod: no client identity on any wire, no keepalive, nothing to wake. The residual is the same allocation gap — per-cluster webhooks cannot see each other's subtree grants.

lakeFS: the best client story — HTTP + per-request SigV4 means the NFS identity class simply does not exist; N clusters are N sets of HTTP clients. The pivot moves to the database: every replica must share ONE KV (cross-region, a KV round-trip sits inside every commit CAS), and two independent installs on one namespace are fenced only at CREATE time (ensureStorageNamespace's dummy object, controller.go:2426) — a birth check, not a live lease; past it, each install's uncommitted GC treats the other's staged objects as garbage. In one line: the hub makes multicluster a *network-identity* problem, the direct modes an *allocation* problem, lakeFS a *database-topology* problem — share one KV or don't.

What moves — score deltas (single → multicluster)

Chart · axis	Column	single → multi	Why
Performance · Stream read / write	hub sys · krb5p	sys 3.5/3 → 3/2.5 krb5p 2/1.5 → 1.5/1	cross-cluster RTT on every RPC; one NIC now shared by N clusters' clients; often an inter-AZ/region billing path too.
Performance · Small files, Hot loops	hub sys · krb5p	sys 1.5/2 → 1/1.5 krb5p 1/1.5 → 0.5/1	the metadata path is already round-trip-bound (create 12.5×, delete 222×); every RTT gets longer.
Performance · Small files	lakeFS	2.5 → 2	only when clusters are remote from the lakeFS service; and if replicas span regions, the KV round-trip sits inside every commit CAS.
Day-2 · Failure shape	hub (both)	2 → 1.5	a hub node death now D-state-hangs pods in every mounting cluster — the blast radius multiplies by topology.
Day-2 · Consumer footprint	hub sys	4 → 3	the cross-cluster operating contract lands on consumers: unique per-node client names, a keepalive, a routable advertiseAddress, peering/SGs. (krb5p stays 1 — already the floor.)
Day-2 · Idle lifecycle & wake	hub (both)	2.5 → 2	idle-suspend kills a live remote mount (suspendWithSessions defaults to suspending anyway), and nothing in a remote cluster can write the wake annotation.

Unchanged by design: every Consistency cell (the hub's coherence is identical for remote clients; the direct modes' contract is already per-subtree snapshots wherever pods run); every FUSE and Lean cell (cross-cluster is cross-pod — the bucket is the rendezvous); krb5p's auth and wire cells (per-user identity survives any network — this is its selling point). And note the **sec=sys security cells are already scored at the multicluster boundary**: their evidence came from the cross-cluster drills (the 1486/1486 SNAT measurement, "nfsClientCIDRs cannot express a client in another cluster"), so they take no further delta here.

Sources: the N-clusters-one-hub drill campaign and flint-lite-architecture p5 (the cross-cluster operating contract), the fleets guide performance tables (single-cluster, same-AZ anchor), and lakeFS controller.go:2426 (the create-time namespace check). Deltas are estimates from the measured single-cluster numbers plus the documented cross-cluster mechanics — not re-measured; treat them as the direction and rough size of the shift.